Report on Programming Task 2

Selma Kiefersauer

March 10, 2026

1 Optical depth

The equation for optical depth for vertical incidence:

$$\tau(z, \lambda, 0) = \sum_j \sigma_j^a(\lambda) \int_z^\infty n_j(z')\, dz' \tag{1}$$

With adding the geometry for non-vertical incidence this is for some angles up to about 60°:

$$\tau(z, \lambda, \chi) = \frac{1}{cos\chi} \sum_j \sigma_j^a(\lambda) \int_z^\infty n_j(z')\, dz' \tag{2}$$

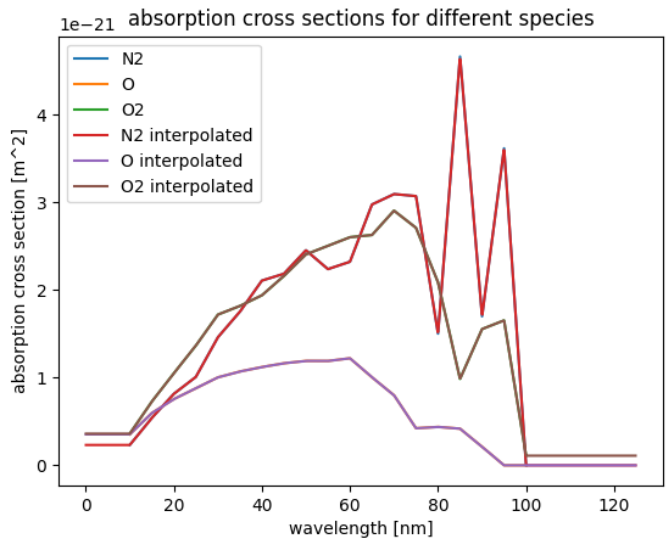
This gets more complicated for even higher incidence angles up to 90°.

$$\tau(z, \lambda, \chi) = \sum_j \sigma_j^a(\lambda) \int_z^\infty n_j(z') \left[1 - \left(\frac{R_E + z_0}{R_E + z'}\right)^2 sin^2\chi_0\right]^{-1/2} dz' \tag{3}$$

For my calculations I used equation (2), as this reduces to equation(1) for vertical incidence anyways as cos(0)=1.

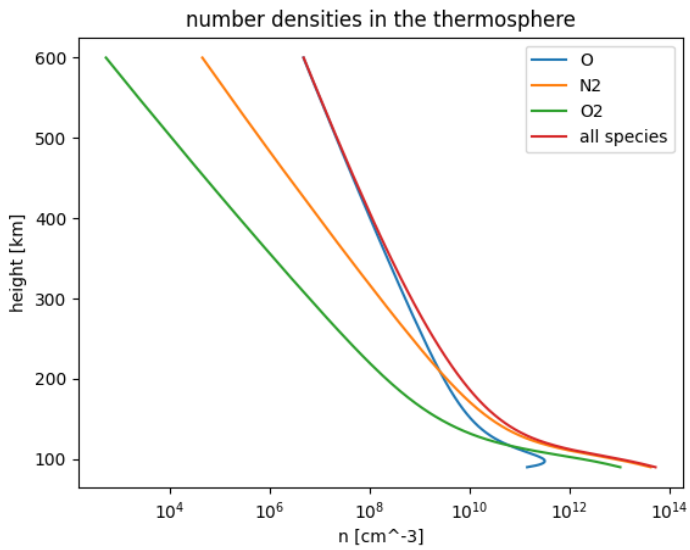
But first the cross sections had to be interpolated to the wavelength resolution in the irradiance file. I used np.interpolate as this gave the best fiting results. But I get 0 for all wavelength after 100nm as there is no more data in the cross section file.

Figure 1: crosssections for different wavelength



Altitude variations of the thermospheric densities, based on the MSIS data file:

Figure 2

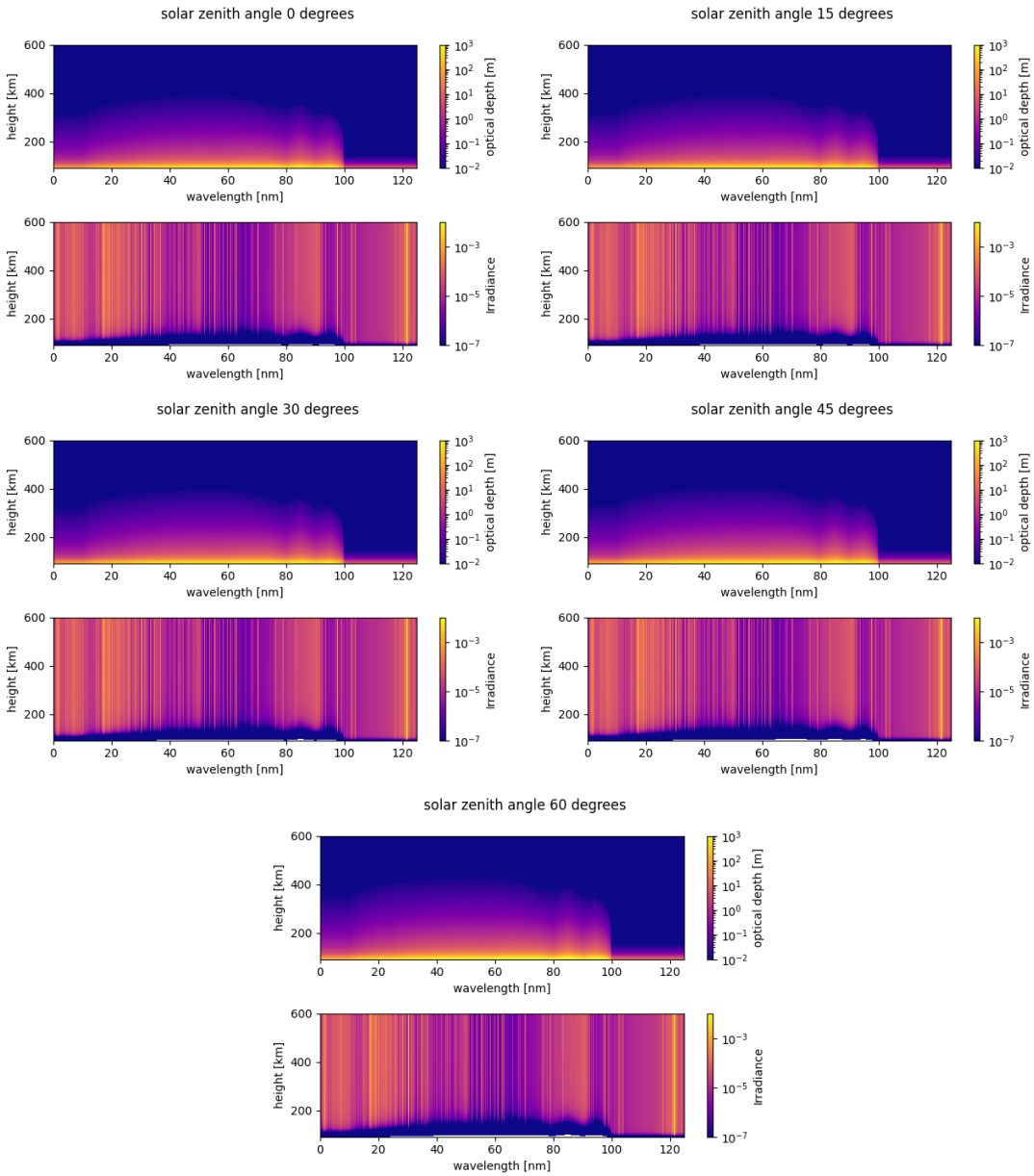


EUV fluxes for different heights and wavelengths can be calculated as:

$$I(z, \lambda) = I^{\infty}(\lambda) exp(-\tau(z, \lambda, \chi)) \tag{4}$$

$I^{\infty}(\lambda)$ is the photon flux at infinity, or at the top of the thermosphere.

Figure 3: Optical depth and EUV fluxes for different angles of incidence.



Python code for the calculations:

```
#relevant angles for Xi
Xi = [0, 15, 30, 45, 60] #[degrees]

#optical depth for different angles of Xi, returns a list over all
                                    wavelength present in input data
#this should work for angles up to 60 degrees
def Tau(n_profiles, nr_of_species, altitude, angle):
    tau = 0
    for i in range(nr_of_species):
        integral = 0
        for z in range( altitude, len(n_data.iloc[:,0]) ):
            integral += n_profiles.iloc[z,i+1] * 1e6 * 1000
```

```
        tau += np.dot( cs[i] , integral )
    return tau * 1/np.cos( angle * (np.pi/180) )

#calculate optical depth for heights in the thermosphere (90-600km
                                    )
#and for different angles of incidence
optical_depth = [] #in [m]
for idx,X in enumerate(Xi):
    T = []
    for m in range(90,600):
        T.append( Tau(n_data, 3, m, X) )
    optical_depth.append(T)

#Irradiance !!what does it mean "/nm" in file?! need to convert
                                    units?!
def Irradiance(opt_depth):
    return np.array(I_data["irradiance"]) * np.exp(-np.array(
                                        opt_depth))


#make plots of optical depth (logarithmic scale for the optical
                                    depth?)
xcoord = I_data["wavelength"]
ycoord = n_data.iloc[91:,0]
for key,ele in enumerate(optical_depth):
    irradiance = Irradiance(ele)
    fig, axs = plt.subplots(2,1)
    plt.suptitle(f"solar zenith angle {Xi[key]} degrees")
    plot1 = axs[0].pcolormesh(xcoord, ycoord, ele, norm=mcolors.
                                        LogNorm(vmin = 1e-2, vmax=
                                        1e3), cmap="plasma")
    plot2 = axs[1].pcolormesh(xcoord, ycoord, irradiance, norm=
                                        mcolors.LogNorm(vmin = 1e-7,
                                        vmax = 1e-2), cmap="plasma")
    cbar1 = fig.colorbar(plot1)
    cbar2 = fig.colorbar(plot2)
    cbar1.set_label("optical depth [m]")
    cbar2.set_label("Irradiance")
    axs[0].set_xlabel("wavelength [nm]")
    axs[0].set_ylabel("height [km]")
    axs[1].set_xlabel("wavelength [nm]")
    axs[1].set_ylabel("height [km]")
    plt.tight_layout()
    plt.show()
```